CS-1204

Object-Oriented Programming (3+1)

Week 01

By

Engr. Muhammad Awais

Lectures : April 14 & April 15 , 2025
Lab: April 18, 2025

Introduction

Engr. Muhammad Awais
Email: awais.rathore@ajku.edu.pk

- Experience
  - Lab Engineer since 2017.
  - Course Instructor PM Kamyab Jawan Program Batch II & Batch III.
  - Software Developer (2013-2017).
  - Freelancing 2017 onwards (1400+ Projects).
  - Software House since 2022 (5 full-time employees hired from DSE).
- Education
  - PhD Software Engineering (In Progress), NUST.
  - MS Software Engineering , ABASYN.
  - B.Sc. Computer Systems Engineering, MUST.

Schedule

- Lecture & Lab:

  Lectures: 32 Labs : 14

- Length :

  Class: 3 hrs. / week
  Lab: 3 hrs. / week

- Day/Time :

  Class: Mon & Tue 10:00 AM– 11:30 PM

  Lab: Friday 10.00 AM – 01:00 PM

- Office Hrs :

  Thursday 10:00 AM – 12:00 PM

Contents

- Introduction to Object Oriented Programming
- Classes and Objects
- Constructors and Destructors
- Encapsulation & Abstraction
- Inheritance and it’s Types
- Function Overriding
- Polymorphism
- Friend Functions and Friend Classes
- File Handling
- Templates
- Exception Handling

Textbooks

#	Title	Author
01	C++ Programming: From Problem Analysis to Program Design, 6th Edition	D.S. Malik
02	Object Oriented Programming in C++ , latest edition.	Robert Lafore

Learning Outcomes

CLO	Statement	Learning Levels	PLO
CLO 1	Understand the fundamental principles of object-oriented programming.	C-2 *Understanding (Level 2)*	PLO 1 *Engineering Knowledge*
CLO 2	Apply object-oriented principles to develop programs	C-3 *Application (Level 3)*	PLO3 Design/Development of Solution
CLO 3 Lab	Develop programs by applying object-oriented principles.	C-3 *Application (Level 3)*	PLO3 Design/Development of Solution

Individual score % to pass CLO = 50 %

Cohort score % to pass CLO = 60%

Evaluation

Activity	Qty
Assignments	02
Quiz	02
Mid Term	01
Final Term	01
Project	01
Labs	14

What do you already know about C++?

What's the biggest C++ program you've written so far?

What comes in your mind when you hear Object Oriented Programming?

What is Object-Oriented Programming??

- Object-Oriented Programming is a programming paradigm that organizes software design around objects, which are instances of classes.
- These objects can contain:
  - **Data** (attributes or properties)
  - **Functions** (methods or behaviors)

Objectives of OOP

- Model real-world entities in code.
- Organize large programs efficiently.
- Improve code reusability, readability, and maintainability.
- Enable modular, scalable, and secure design.

Why Do We Need OOP?

- **Procedural programming** becomes complex for large systems.
- Data and functions are loosely connected.
- Difficult to maintain and debug.
- No data hiding — all functions can access global data.

OOP solves these issues with:

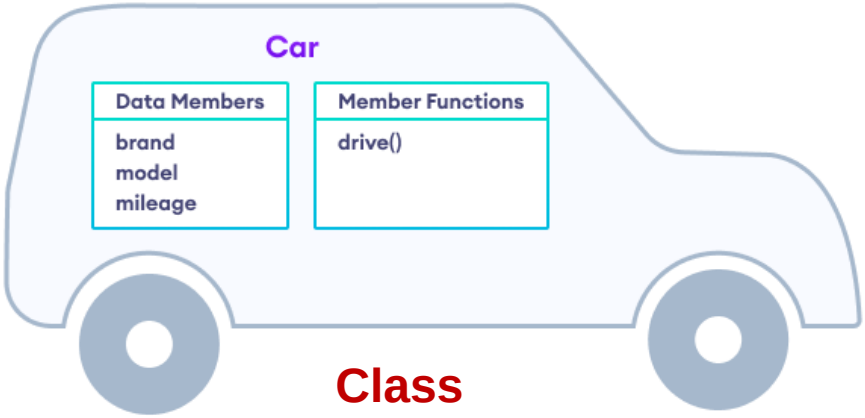
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

C++ Class

- A class is a user-defined data type.
- It is a blueprint for the object.
- Everything in C++ is associated with classes and objects, along with its attributes and methods.
- Attributes and methods are basically variables and functions that belongs to the class. These are often referred to as "class members".

Example of a class & object

- We can think of a class as the technical design (prototype) of a car. It contains all the details about the brand, model, mileage, etc. We can then build different cars based on these descriptions.
- Each distinct car is an object.
- Objects are also called as an instance of class.



1. Toyota
2. Kia
3. Audi

Object of a class Car

Create a class in C++

To create a class, use the `class` keyword:

Example

Create a class called "`MyClass`":

```
class MyClass {       // The class
  public:             // Access specifier
    int myNum;        // Attribute (int variable)
    string myString;  // Attribute (string variable)
};
```

Example explained

- The `class` keyword is used to create a class called `MyClass`.
- The `public` keyword is an **access specifier**, which specifies that members (attributes and methods) of the class are accessible from outside the class. You will learn more about access specifiers later.
- Inside the class, there is an integer variable `myNum` and a string variable `myString`. When variables are declared within a class, they are called **attributes**.
- At last, end the class definition with a semicolon `;`.

Create an Object

In C++, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class name, followed by the object name.

To access the class attributes (`myNum` and `myString`), use the dot syntax (`.`) on the object:

Example

Create an object called "`myObj`" and access the attributes:

```
class MyClass {       // The class
  public:             // Access specifier
    int myNum;        // Attribute (int variable)
    string myString;  // Attribute (string variable)
};

int main() {
  MyClass myObj;  // Create an object of MyClass

  // Access attributes and set values
  myObj.myNum = 15;
  myObj.myString = "Some text";

  // Print attribute values
  cout << myObj.myNum << "\n";
  cout << myObj.myString;
  return 0;
}
```

Example of a class with multiple objects

Example

```
// Create a Car class with some attributes
class Car {
  public:
    string brand;
    string model;
    int year;
};

int main() {
  // Create an object of Car
  Car carObj1;
  carObj1.brand = "BMW";
  carObj1.model = "X5";
  carObj1.year = 1999;

  // Create another object of Car
  Car carObj2;
  carObj2.brand = "Ford";
  carObj2.model = "Mustang";
  carObj2.year = 1969;

  // Print attribute values
  cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
  return 0;
}
```

Member Functions of a Class - Methods

- Methods are **functions** that belongs to the class.
- There are two ways to define functions that belongs to a class:
  - Inside class definition
  - Outside class definition
- You access methods just like you access attributes; by creating an object of the class and using the dot syntax (.)

Inside Example

```
class MyClass {        // The class
  public:              // Access specifier
    void myMethod() {  // Method/function defined inside the class
      cout << "Hello World!";
    }
};

int main() {
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

Member Function Outside Class Definition

- To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by specifiying the name of the class, followed the scope resolution :: operator, followed by the name of the function:

Outside Example

```
class MyClass {        // The class
  public:              // Access specifier
    void myMethod();   // Method/function declaration
};

// Method/function definition outside the class
void MyClass::myMethod() {
  cout << "Hello World!";
}

int main() {
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

Example of class with data members and member functions

```cpp
#include <iostream>
using namespace std;
class Car {
  public:
    // class data
    string brand, model;
    int mileage = 0;
    // class function to drive the car
    void drive(int distance) {
        mileage += distance;
    }
    // class function to print variables
    void show_data() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Distance driven: " << mileage << " miles" << endl;
    }
};

int main() {
    // create an object of Car class
    Car my_car;
    // initialize variables of my_car
    my_car.brand = "Honda";
    my_car.model = "Accord";
    my_car.drive(50);
    // display object variables
    my_car.show_data();
    return 0;
}
```

```
E:\C++\oop1.exe

Brand: Honda
Model: Accord
Distance driven: 50 miles

--------------------------------
Process exited after 0.1024 seconds with return value 0
Press any key to continue . . .
```

Task 1: Student Record System (Functions Inside Class)

Create a class Student with the following attributes:

- string name
- int rollNumber
- float marks

Define the following member functions inside the class:

- getData() – to take input from the user
- displayData() – to display student information

Create an object and test the functionality.

Task 2: Rectangle(Functions Outside Class)

Create a class Rectangle with attributes:

- int length
- int width

Define the following member functions outside the class:

- setDimensions() – to input length and width
- area() – to calculate area
- perimeter() – to calculate perimeter

Create an object and test the functionality.

Task 3: Voting Eligibility

Create a class Voter with attributes:

- string name
- int age

Define the following member functions outside the class:

- isEligible() – to check the eligibility of the voter. (Only voter age > 18 is eligible). This method should return true of false based on the age of the person.
- Based on what the isEligible() method returns show an appropriate message to the user in main function.

Create an object and test the functionality.

Task 4: Temperature Convertor

Create a class named Temperature that converts temperature from Celsius to Fahrenheit. The class should have:

- A public data member to store temperature in Celsius.
- A function to convert the temperature using the formula.

$$\text{Fahrenheit} = (\text{Celsius} \times \frac{9}{5}) + 32$$

- A function to display the converted value.

Create an object and test the functionality.

Task 5: Calculator(Functions Outside Class)

Create a class Calculator with attributes:

- float num1
- float num2

Define the following member functions inside or outside the class:

- add()
- subtract()
- multiply()
- divide()

Allow the user to input two numbers and choose an operation to perform.

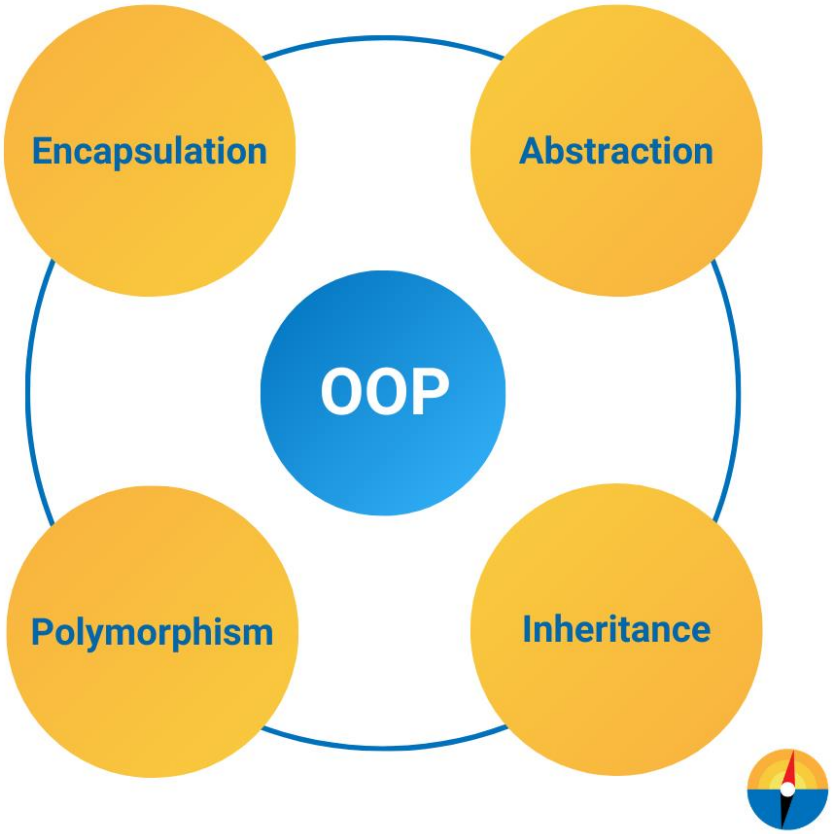
Task 5: Separate Header Files

Reorganize all the programs by creating separate header files to improve code structure and readability. Include these header files in the main program and demonstrate the use of objects accordingly.

Rubrics

Criteria	Excellent (3)	Good (2)	Need Improvement (1)	Poor (0)
Class Definition & Object Usage	Correct class structure with object creation; proper syntax.	Minor syntax issues but structure is mostly correct.	Major issues in class or object usage.	Not attempted / completely incorrect.
Member Variables	Relevant attributes defined clearly with proper data types.	Attributes defined with minor issues.	Attributes poorly defined or unclear.	No meaningful use of member variables.
Member Functions (Inside/Outside)	Functions defined and used correctly as per instruction.	Functions defined with small syntax/placement errors.	Poor function structure or logic missing.	No functions defined or irrelevant code.
Input/Output & Execution	Takes input and displays output correctly and clearly.	Input/output mostly works with small issues.	Output unclear or partially working.	Code does not run or no input/output.

Pillars of Object-Oriented Programming



Encapsulation in C++

What is Encapsulation?

- Encapsulation is the process of bundling data and methods into a single unit (class).
- It hides internal data from outside interference.
- Data is accessed and modified only through public methods.

Why we use Encapsulation?

- Protect data from accidental or unauthorized changes.
- Control how data is accessed or modified.
- Add validation logic in one place. Simplify future maintenance..

Encapsulation in C++ (cont…)

Real-Life Analogies

- **ATM Machine:** You can’t access the balance directly. You use buttons (functions).
- **Car:** You don't touch the engine directly. You use the accelerator, brake, and steering.
- **Medicine Capsule:** You don’t see the ingredients—only the final product..

Encapsulation in C++ (cont…)

Achieving Encapsulation

- Make data members **private**.
- Provide public **getter and setter** methods.
- **Validate** data inside setters.

```
class Student {
private:
    string name;
    int age;

public:
    void setName(string n) {
        if (!n.empty()) {
            name = n;
        } else {
            cout << "Invalid name. Name cannot be empty." << endl;
        }
    }

    void setAge(int a) {
        if (a > 0 && a < 120) {
            age = a;
        } else {
            cout << "Invalid age. Age must be between 1 and 119." << endl;
        }
    }

    string getName() { return name; }
    int getAge() { return age; }
};
```

Encapsulation in C++ (cont…)

Benefits

- Data members are protected.
- Setters perform validation.
- Code is cleaner, more secure.

```
class Student {
private:
    string name;
    int age;

public:
    void setName(string n) {
        if (!n.empty()) {
            name = n;
        } else {
            cout << "Invalid name. Name cannot be empty." << endl;
        }
    }

    void setAge(int a) {
        if (a > 0 && a < 120) {
            age = a;
        } else {
            cout << "Invalid age. Age must be between 1 and 119." << endl;
        }
    }

    string getName() { return name; }
    int getAge() { return age; }
};
```

Encapsulation in C++ (cont…)

Common Mistakes To Avoid

- Making variables public just to "make it work".
- Skipping validation in setters.
- Writing getters/setters without protecting data.

Encapsulation in C++ (cont…)

Another Example

```
class BankAccount {
private:
    double balance;

public:
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            cout << "Invalid deposit amount." << endl;
        }
    }

    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        } else {
            cout << "Invalid withdraw amount or insufficient funds." << endl;
        }
    }

    double getBalance() { return balance; }
};
```

Task 5: Encapsulation

Create a Product class with proper encapsulation.

Private data members:

- string name
- double price
- int quantity

Public Methods:

- void setName(string n) - Validate non-empty name.
- void setPrice(double p) - Validate price > 0.
- void setQuantity(int q) - Validate quantity >= 0
- Getters for all fields

Test Code:

```
int main() {
    Product p;
    p.setName(""); // Should print error message
    p.setPrice(-50); // Should print error message
    p.setQuantity(-2); // Should print error message

    p.setName("Laptop");
    p.setPrice(1200.50);
    p.setQuantity(5);

    cout << "Product: " << p.getName() << endl;
    cout << "Price: $" << p.getPrice() << endl;
    cout << "Quantity: " << p.getQuantity() << endl;
    return 0;
}
```

REFERENCES

- https://www.w3schools.com/cpp/cpp_oop.asp
- https://www.programiz.com/cpp-programming/oop